Using automation dashboard

This guide describes how to install and configure automation dashboard to evaluate automation usage across your environments and the savings associated with it.

Providing feedback on Red Hat documentation

If you have a suggestion to improve this documentation, or find an error, you can contact technical support at https://access.redhat.com to open a request.

1. View key usage metrics with automation dashboard

By effectively using automation dashboard, you can gain valuable insights into your Ansible Automation Platform usage and drive continuous improvement in your automation practices.

1.1. About the automation dashboard

The automation dashboard utility is a web-based container application that provides key metrics related to **job execution**, **efficiency**, and the **value derived from automation**.

Automation dashboard uses automation metrics to supply automation usage data from Ansible Automation Platform. This data helps you compare the cost of performing tasks manually to the cost of performing tasks through automation, allowing you to show what savings are achievable through automation.

1.1.1. Key benefits

Automation dashboard helps you:

- **Get a clear overview** of the automation occurring in your environment.
- **Track metrics** such as time saved and errors reduced, to quantify the benefits of automation.
- **Analyze job execution times and failure rates** to pinpoint areas for automation improvement.
- **Use the generated data** to make informed decisions about automation strategy, resource allocation, and prioritization of automation projects.

1.2. Installing automation dashboard

Install automation dashboard to collect and analyze key metrics related to job execution, efficiency, and automation savings across your Ansible Automation Platform deployments.

Prerequisites

- One of the following tested configurations:
  - RHEL 9 or RHEL 10 on x86 or ARM-based physical or virtual hosts.
  - With an external database: PostgreSQL v15 database.

IMPORTANT	Do not install automation dashboard on the same host(s) as Ansible Automation Platform.

- Automation dashboard installation has been tested with the following configuration:
  - 80 GB hard drive (depending on data growth)
  - 4 vCPUs x 16 GB RAM
  - Disk IOPS - 3000
  - Handle up to 10,000 jobs/month and 47M summaries/month
  - Connects to three Ansible Automation Platform deployments of the same version
- Access to *baseos* and *Ansible Automation Platformstream* repository packages for the RHEL 9 or RHEL 10 host.
- A non-root login account to the RHEL 9 or RHEL 10 host for installation.
  - The login account requires `sudo` access to root.
  - By default, we use the `$HOMEDIR` of the user account.
- URL details for access to your Ansible Automation Platform instances.
- An Ansible Automation Platform OAuth2 token, which is used for communication between the Ansible Automation Platform instances and automation dashboard.
- Access to download the installation bundle providing installation components for the automation dashboard.
- Open firewall access to allow for bidirectional communication between Ansible Automation Platform instances and the automation dashboard.
  - This includes HTTPS/443 (or your Ansible Automation Platform configured port) from the dashboard to the Ansible Automation Platform instance(s).
  - Port 8447 is the default ingress port for the automation dashboard. This port can be configured during installation.
  - RHEL firewall ports that might block 5432 to PostgreSQL.
- A supported version of `ansible-core` installed on supported RHEL versions.

Procedure

1. Download the latest installer tar file from access.redhat.com. Navigate to Downloads > Red Hat Ansible Automation Platform Product Software.
2. Copy the installation source file to your RHEL 9 or RHEL 10 host.
3. Extract the installation source. This will require ~500Mb. of disk space. Throughout this example we will use the ec2-user home directory: `/home/<username>`.

```
tar -xzvf ansible-automation-dashboard-containerized-setup-bundle-0.1-x86_64.tar.gz
cd ansible-automation-dashboard-containerized-setup/
```

4. Install the required `ansible-core` package.

```
cd ansible-automation-dashboard-containerized-setup
sudo dnf install ansible-core
```

5. Create an application `client_id/client_secret` in your Ansible Automation Platform instance:
   a. Create an OAuth2 application using the following steps :
      i. **For Ansible 2.4**:
         - Navigate to https://AAP_CONTROLLER_FQDN/#/applications
      ii. **For Ansible 2.5 and 2.6**:
         - Navigate to https://AAP_GATEWAY_FQDN/access/applications
      iii. Add the following information:
         - **Name**: automation-dashboard-sso
         - **Authorization grant type**: authorization-code
         - **Organization**: Default
         - **Redirect URIs**: https://AUTOMATION_DASHBOARD_FQDN/auth-callback
         - **Client type**: Confidential

NOTE	The values for **Name**, **Organization**, and HTTPS port number for Ansible Automation Platform are configurable. The examples provided in this document assume use of port 443.

   b. Save the `client_id` and `client_secret information` inputs into the inventory file.
   c. Create an Ansible Automation Platform access token:
      i. Navigate to https://AAP_GATEWAY_FQDN/#/users/<id>/tokens, and create a token using the following information:
         - OAuth application: automation-dashboard-sso
         - Scope: read
      ii. Store the access token and refresh token values. `clusters.yaml` uses these tokens.
6. Copy the example inventory and change it before running the installation program.

```
cp -i inventory.example inventory
vi inventory
```

IMPORTANT	◦ This is an example tested inventory containing default values for

Ansible Automation Platform 2.4, 2.5, and 2.6.

- You must change the following values to use this inventory configuration in your environment:
  - Change the RHEL 9 or RHEL 10 host occurrences from `host.example.com` to your FQDN host
  - Change the phrase `TODO` to match your passwords within all `_admin_password` or `_pg_password` values.
  - Change the `dashboard_pg_host` value to the IP address or DNS name of the database server.
- For more information, see the Inventory variables section of this document.

```
# This is our automation dashboard front-end application
[automationdashboard]
host.example.com ansible_connection=local

# These are required vars for the installation and should not be removed
[automationdashboard:vars]
# Configure AAP OAuth2 authentication.
# aap_auth_provider_name - name as shown on login page.
aap_auth_provider_name=Ansible Automation Platform
# aap_auth_provider_protocol - http or https
aap_auth_provider_protocol=https
# AAP version - 2.4, 2.5 or 2.6
aap_auth_provider_aap_version=2.5
# aap_auth_provider_host - AAP IP or DNS name, with optional port
aap_auth_provider_host=my-aap.example.com
# aap_auth_provider_check_ssl - enforce TLS check or not.
aap_auth_provider_check_ssl=true
# aap_auth_provider_client_id and aap_auth_provider_client_secret -
# they are obtained from AAP when OAuth2 application is created in AAP.
aap_auth_provider_client_id=TODO
aap_auth_provider_client_secret=TODO


# Specify amount of old data to synchronoize after installation.
# The initial_sync_days=N requests N days of old data, counting from "today".
# The initial_sync_since requests data from the specified data until "today".
# If both are specified, the initial_sync_since will be used.
initial_sync_days=1
# initial_sync_since=2025-08-08

# Hide warnings when insecure https request are made.
# Use this if your AAP uses self-signed TLS certificate.
# show_urllib3_insecure_request_warning=False

# Force clean install-like
```

```
# dashboard_update_secret=true

# HTTP/HTTPS settings
# nginx_disable_https=true
# Change nginx_http_port or nginx_https_port if you want to access dashboard on a
different TCP port.
# nginx_http_port=8083
# nginx_https_port=8447
# TLS certificate configuration
# The dashboard_tls_cert needs:
#   - contain server certificate, intermediate CA certificates and root CA
certificate.
#   - the server certificate must be the first one in the file.
# dashboard_tls_cert=/path/to/tls/dashboard.crt
# dashboard_tls_key=/path/to/tls/dashboard.key

# Enable Django DEBUG.
# django_debug=True

[database]
host.example.com ansible_connection=local
[redis]
host.example.com ansible_connection=local

[all:vars]
redis_mode=standalone

[all:vars]
postgresql_admin_username=postgres
postgresql_admin_password=TODO

# registry_username=
# registry_password=
# AAP Dashboard - mandatory
# --------------------------
dashboard_pg_containerized=True
dashboard_admin_password=TODO
# The value of dashboard_pg_host needs to resolve to an IP address of the database.
# It cannot be localhost or 127.0.0.1.
# It can be IP address of the server.

dashboard_pg_host=host.example.com
dashboard_pg_username=aapdashboard
dashboard_pg_password=TODO
dashboard_pg_database=aapdashboard

bundle_install=true
#
# Relevant if bundle_install=false
# Set container_image_update to True to fetch most recent container images from
registry.
```

```
# container_image_update=True
#
# Relevant if bundle_install=true
# <full path to the bundle directory>
bundle_dir='{{ lookup("ansible.builtin.env", "PWD") }}/bundle'
# AAP Dashboard - optional
# -------------------------
# Set to True to expose Django admin page.
# nginx_dashboard_admin_exposed=False
```

7. Install the required Ansible collections. You must complete this step to prevent module resolution errors during the installation.

```
ansible-galaxy collection install -r requirements.yml
```

8. Run the installation program.

```
ansible-playbook -i inventory ansible.containerized_installer.dashboard_install
--ask-become-pass
```

Verification

For reference, see the following example output:

```
PLAY RECAP
*********************************************************************************
*****************************************************
ec2-54-147-26-173.compute-1.amazonaws.com : ok=126  changed=51   unreachable=0
failed=0    skipped=42   rescued=0    ignored=0
localhost                  : ok=12   changed=0    unreachable=0    failed=0
skipped=9    rescued=0    ignored=0
```

Alternative configurations are possible (for example, the database for automation dashboard can be set on a different host). This requires additional changes to variables in the inventory file. Consult the Inventory variables section of this document for available variables.

1.3. Integrating automation dashboard with your Ansible Automation Platform

Integrate your Ansible Automation Platform instances into the automation dashboard configuration to collect and visualise data and gain insights into your automation.

Prerequisites

- You have installed automation dashboard.
- You have verified that automation dashboard is running on HTTPS port 8447 on your Red Hat

Enterprise Linux host.

NOTE	◦ This verification requires your Ansible Automation Platform login details. ◦ Port 8447 is enabled by default, but this is configurable.

Procedure

1. Configure a personal access token. For more information, see Configuring access to external applications with token-based authentication.
2. Create or update the `clusters.yaml` file. You must include the `access_token`, `refresh_token`, `client_id`, and `client_secret` to enable persistent authentication and automatic token rotation.

```
clusters:
  - protocol: https
    address: <aap_controller_url>
    port: 443
    access_token: <access_token_string>
    refresh_token: <refresh_token_string>
    client_id: <client_id_string>
    client_secret: <client_secret_string>
    verify_ssl: false
    name: <unique_cluster_name>
```

Configuring TLS verification

When configuring the `verify_ssl` parameter, choose the setting that matches your Ansible Automation Platform certificate type:

- **Commercial certificates:** Set `verify_ssl: true`.
- **Self-signed certificates:** Set `verify_ssl: false`.

NOTE	The automation dashboard cannot verify self-signed certificates against a custom Certificate Authority (CA).

3. You can add one or more Ansible Automation Platform instances (of the same Ansible Automation Platform version) into the automation dashboard configuration for pulling and combining data by using the following:

NOTE	If you only have one Ansible Automation Platform instance, then remove the second entry.

```
---
clusters:
  - protocol: https         <--- Normally https
    address: my-aap.example.com  <--- Can use IP or FQDN without http(s)://
    port: 443               <--- Normally 443
    access_token: sampleToken   <--- Your preconfigured Ansible Automation Platform
```

```
read access token
  Platform read access token
    refresh_token: myRefreshToken
    client_id: myClientID
    client_secret: myClientSecret
    verify_ssl: false       <--- Can be used when using self signed certs
    sync_schedules:
      - name: Every 5 minutes sync
        rrule: DTSTART;TZID=Europe/Ljubljana:20250630T070000
FREQ=MINUTELY;INTERVAL=5
        enabled: true
 # If there is one Ansible Automation Platform instance, then remove the second
entry below.
  # Alternately. If there is more than one AAP instance to connect, copy additional
entries.
  - protocol: https
    address: aap2.example.com
    port: 443
    access_token: WRn2swiqg5spEwUndDkrJoCeg4Qwuw
    verify_ssl: true
    sync_schedules:
      - name: Every 5 minutes sync
        rrule: DTSTART;TZID=Europe/Ljubljana:20250630T070000
FREQ=MINUTELY;INTERVAL=5
       enabled: true
```

NOTE The `access_token`, `refresh_token`, and `client_secret` are stored in the automation dashboard database. These values are encrypted for security.

4. Load and activate your automation dashboard configuration:

```
podman cp clusters.yaml automation-dashboard-web:/
podman exec -it automation-dashboard-web /venv/bin/python manage.py setclusters
/clusters.yaml
```

NOTE

Token Rotation: The automation dashboard rotates tokens internally for security but does not update your local `clusters.yaml` file. If you must re-run the `setclusters` command later, your defined tokens might be expired.

To retrieve the current valid tokens, run the following command and save the output to a new file:

```
podman exec -it automation-dashboard-web /venv/bin/python manage.py
getclusters --decrypt > clusters_current.yaml
```

1.3.1. Synchronizing data to automation dashboard

Synchronize data from your connected Ansible Automation Platform clusters to the automation dashboard to view the latest automation metrics.

Prerequisites

- You have installed automation dashboard.
- You have configured the `clusters.yaml` file with your platform details.

Procedure

1. Identify the running automation dashboard container:

```
$ podman ps | grep automation-dashboard-web
```

2. Run the synchronization command using one of the following methods:

   - **Interactive mode:** Run `syncdata` without arguments. You must confirm the synchronization request when prompted.

```
$ podman exec -it automation-dashboard-web /venv/bin/python manage.py syncdata
```

   - **Noninteractive mode:** Use the `--since` and `--until` flags to define the date range. This method bypasses the user prompt and is required for automated scripts or `cron` jobs.

```
$ podman exec -it automation-dashboard-web /venv/bin/python manage.py syncdata
--since=2024-01-01 --until=2024-06-30
```

Verification

1. Verify that the terminal displays the success message: `Successfully created Sync task for Cluster <cluster_url>`.
2. Refresh automation dashboard in your browser to view the updated metrics.

1.3.2. Verifying cluster access tokens

After you configure and load your cluster data, verify the stored access tokens for debugging purposes.

Procedure

1. Use the `getclusters` management command with the `--decrypt` option to display the stored `access_token` and `refresh_token` in plain text.
2. Run the following command inside the `automation-dashboard-web` container:

```
podman exec -it automation-dashboard-web /venv/bin/python ./manage.py getclusters
```

```
--decrypt
```

3. Review the output to confirm that the stored tokens are correct and up-to-date.

```
clusters:
  - protocol: https
    address: my-aap.example.com
    port: 443
    access_token: sampleToken
    refresh_token: myRefreshToken
    client_id: myClientID
    client_secret: myClientSecret
    verify_ssl: false
    sync_schedules:
      - name: Every 5 minutes sync
        rrule: DTSTART;TZID=Europe/Ljubljana:20250630T070000
FREQ=MINUTELY;INTERVAL=5
        enabled: true
```

NOTE Displaying the encrypted `access_token` and `refresh_token` in plain text for debugging requires the `--decrypt` flag. Do not use this command on unsecured systems.

You can write output produced by `./manage.py getclusters --decrypt` to a file `clusters.yaml` and use it as input for `./manage.py setclusters clusters.yaml`.

Verification

If you come across error messages during installation, consult the following table:

Issue	Possible Cause	Solution
401 error	This is an unauthorized access message indicating authentication errors such as wrong credentials or tokens.	Verify that your access token is correct in `clusters.yaml`

401 error	A temporary 401 error is expected behavior when the token expires, followed immediately by trying to refresh.	If the automatic token refresh fails (for example, due to invalid `client_secret` or `refresh_token`), use the `getclusters --decrypt` command to manually verify that the credentials stored in the database match those in your source `clusters.yaml` file. If they do not match, re-run the `setclusters` command with the correct configuration. You can only use the refresh token once. If you need to execute `setclusters` because of invalid access token, create new access and refresh tokens, and use them in your source `clusters.yaml`.
404 error	This is a “not found” message indicating that something is not configured correctly or pointing to the correct endpoint.	Verify that your Ansible Automation Platform instance URLs used in `clusters.yaml` are correct.

A successful installation should be running the following three container services:

```
podman ps --all --format "{{.Names}}"

postgresql
automation-dashboard-task
automation-dashboard-web
```

You can check your container logs by running the following:

```
journalctl CONTAINER_NAME=container (where container equals one of postgresql
automation-dashboard-task or automation-dashboard-web)

For example:
journalctl CONTAINER_NAME=automation-dashboard-task
May 22 13:02:07 automation-dashboard automation-dashboard-task[1607]: [wait-for-
migrations-dashboard.sh] Waiting for database migrations...
May 22 13:02:07 automation-dashboard automation-dashboard-task[1607]: [wait-for-
migrations-dashboard.sh] Attempt 1
May 22 13:02:10 automation-dashboard automation-dashboard-task[1607]: INFO 2025-05-22
13:02:10,636 periodic 2 140568371550016 Starting sync task.
May 22 13:02:10 automation-dashboard automation-dashboard-task[1607]: INFO 2025-05-22
13:02:10,636 periodic 2 140568371550016 Retrieving clusters inform>
May 22 13:02:10 automation-dashboard automation-dashboard-task[1607]: INFO 2025-05-22
13:02:10,747 periodic 2 140568371550016 Retrieved 1 clusters.
May 22 13:02:10 automation-dashboard automation-dashboard-task[1607]: INFO 2025-05-22
13:02:10,761 periodic 2 140568371550016 Retrieving data from clust>
```

```
May 22 13:02:10 automation-dashboard automation-dashboard-task[1607]: INFO 2025-05-22
13:02:10,761 connector 2 140568371550016 Checking Ansible Automation Platform version
at h>
May 22 13:02:10 automation-dashboard automation-dashboard-task[1607]: INFO 2025-05-22
13:02:10,761 connector 2 140568371550016 Checking if is Ansible Automation Platform
2.5 at>
May 22 13:02:10 automation-dashboard automation-dashboard-task[1607]: INFO 2025-05-22
13:02:10,761 connector 2 140568371550016 Pinging api https://ec2-3>
May 22 13:02:10 automation-dashboard automation-dashboard-task[1607]: INFO 2025-05-22
13:02:10,761 connector 2 140568371550016 Executing GET request to >
May 22 13:02:13 automation-dashboard automation-dashboard-task[1607]: ERROR 2025-05-22
13:02:13,820 connector 2 140568371550016 GET request failed with >
May 22 13:02:13 automation-dashboard automation-dashboard-task[1607]: INFO 2025-05-22
13:02:13,821 connector 2 140568371550016 Checking if is Ansible Automation Platform
2.4 at>
May 22 13:02:13 automation-dashboard automation-dashboard-task[1607]: INFO 2025-05-22
13:02:13,821 connector 2 140568371550016 Pinging api https://ec2-3>
May 22 13:02:13 automation-dashboard automation-dashboard-task[1607]: INFO 2025-05-22
13:02:13,821 connector 2 140568371550016 Executing GET request to >
May 22 13:02:16 automation-dashboard automation-dashboard-task[1607]: ERROR 2025-05-22
13:02:16,892 connector 2 140568371550016 GET request failed with ...
```

The following log snippet shows a successful token refresh:

NOTE	This log snippet omits timestamps and hostname for brevity.

Example

```
journalctl CONTAINER_NAME=automation-dashboard-task
INFO Checking if is AAP 2.5 ... 2.6 at https://app.example.com:443
INFO Pinging api https://app.example.com:443/api/gateway/v1/ping/
INFO Detected AAP version AAP 2.6 at https://app.example.com:443
INFO Executing GET request to
https://app.example.com:443/api/controller/v2/organizations/?page_size=100&page=1
ERROR GET request failed with status 401
INFO Token refresh POST request succedded with status 201
ERROR GET after reauth response.status_code=200
INFO Executing GET request to
https://app.example.com:443/api/controller/v2/job_templates/?page_size=200&page=1
Executing GET request to
https://app.example.com:443/api/controller/v2/jobs/?page_size=100&page=1&order_by=fini
shed&finished__gt=2025-10-23T13:01:09.768681Z
```

Check how the services are running by using `systemd`:

```
systemctl status --user

 automation-dashboard
```

```
    State: running
    Units: 76 loaded (incl. loaded aliases)
     Jobs: 0 queued
   Failed: 0 units
    Since: Thu 2025-05-22 13:02:07 UTC; 22min ago
  systemd: 252-51.el9
   CGroup: /user.slice/user-1000.slice/user@1000.service
           ├─app.slice
           │ ├─automation-dashboard-task.service
           │ │ └─1607 /usr/bin/conmon --api-version 1 -c
84e46532e8ca31b0cadb037479289d030103aa01b7a1591e62b83b17f031e47d -u
84e46532e8ca31b0cadb037479>
           │ ├─automation-dashboard-web.service
           │ │ └─1608 /usr/bin/conmon --api-version 1 -c
d060f3e3fb2b4c4c5c588149253beed83c78ccc9c9a8c1bf4c96157142a210dc -u
d060f3e3fb2b4c4c5c58814925>
           │ ├─dbus-broker.service
           │ │ ├─1621 /usr/bin/dbus-broker-launch --scope user
           │ │ └─1624 dbus-broker --log 4 --controller 9 --machine-id
612db98503014199bfd8c788c8d3da58 --max-bytes 100000000000000 --max-fds 2500000000>
           │ └─postgresql.service
           │   └─1614 /usr/bin/conmon --api-version 1 -c
eec61745cb6fc3a89a4f7475d7ef63b5899699157d943c2f16a3243311927bef -u
eec61745cb6fc3a89a4f7475d7>
           ├─init.scope
           │ ├─1093 /usr/lib/systemd/systemd --user
           │ └─1128 "(sd-pam)"
           └─user.slice
             ├─libpod-
84e46532e8ca31b0cadb037479289d030103aa01b7a1591e62b83b17f031e47d.scope
             │ └─container
             │   ├─1619 /usr/bin/dumb-init -- /usr/bin/launch_dashboard_task.sh
             │   └─1681 /venv/bin/python periodic.py
             ├─libpod-
d060f3e3fb2b4c4c5c588149253beed83c78ccc9c9a8c1bf4c96157142a210dc.scope
             │ └─container
             │   ├─1617 /usr/bin/dumb-init -- /usr/bin/launch_dashboard_web.sh
             │   ├─1646 /usr/bin/python3.9 /usr/local/bin/supervisord -c
/etc/supervisord_dashboard_web.conf
             │   ├─1877 /bin/bash /usr/local/bin/stop-supervisor
             │   ├─1878 "nginx: master process nginx -g daemon off;"
             │   ├─1879 /venv/bin/uwsgi /etc/tower/uwsgi.ini
             │   ├─1880 "nginx: worker process"
             │   ├─1881 "nginx: worker process"
             │   ├─1882 "nginx: worker process"
             │   ├─1883 "nginx: worker process"
             │   ├─1884 /venv/bin/uwsgi /etc/tower/uwsgi.ini
             │   ├─1885 /venv/bin/uwsgi /etc/tower/uwsgi.ini
             │   ├─1886 /venv/bin/uwsgi /etc/tower/uwsgi.ini
             │   ├─1887 /venv/bin/uwsgi /etc/tower/uwsgi.ini
             │   └─1888 /venv/bin/uwsgi /etc/tower/uwsgi.ini
```

```
              ├──libpod-
eec61745cb6fc3a89a4f7475d7ef63b5899699157d943c2f16a3243311927bef.scope
              │  └──container
              │    ├──1623 postgres
              │    ├──1869 "postgres: logger "
              │    ├──1871 "postgres: checkpointer "
              │    ├──1872 "postgres: background writer "
              │    ├──1873 "postgres: walwriter "
              │    ├──1874 "postgres: autovacuum launcher "
              │    ├──1875 "postgres: stats collector "
              │    ├──1876 "postgres: logical replication launcher "
              │    └──1889 "postgres: aapdashboard aapdashboard 172.31.28.99(39338)
idle"
              └──podman-pause-b6c4e853.scope
                └──1359 catatonit -P
```

1.4. Upgrading automation dashboard

Upgrade your automation dashboard installation to the latest version. This process involves running the installation program and updating your cluster configuration with new authentication requirements.

NOTE For more information about recent updates to automation dashboard, see What's new: Upgrades for automation dashboard.

Procedure

1. Download the latest installation bundle from access.redhat.com. Navigate to Downloads > Red Hat Ansible Automation Platform Product Software.
2. Extract the bundle to a new directory.
3. Copy the `inventory` file from your previous installation directory to the new directory.
4. Edit the `inventory` file to include the mandatory Redis configuration. You must add a `[redis]` group and define the `redis_mode` variable:

   NOTE The inventory file lines prefixed with + must be added when upgrading your automation dashboard.

```
 [database]
host.example.com ansible_connection=local

+[redis]
+host.example.com ansible_connection=local
+
 [all:vars]
+redis_mode=standalone
+
 postgresql_admin_username=postgres
```

```
postgresql_admin_password=TODO
# registry_username=
```

5. Run the installation playbook from the new directory:

```
ansible-galaxy collection install -r requirements.yml
ansible-playbook -i inventory ansible.containerized_installer.dashboard_install
--ask-become-pass
```

6. Update your `clusters.yaml` file. You must add the `refresh_token`, `client_id`, and `client_secret` variables to your existing cluster configurations.

NOTE	For instructions on obtaining these values, see Integrating automation dashboard with your Ansible Automation Platform.

7. Apply the updated configuration to the dashboard:

```
podman cp clusters.yaml automation-dashboard-web:/
podman exec -it automation-dashboard-web /venv/bin/python manage.py setclusters
/clusters.yaml
```

Verification

1. Retrieve the current cluster configuration:

```
podman exec -it automation-dashboard-web /venv/bin/python ./manage.py getclusters
--decrypt
```

2. Verify that the output displays the content from your `clusters.yaml` file, including the `access_token`, `refresh_token`, `client_id`, and `client_secret` fields.

1.4.1. Troubleshooting synchronization failures

If new jobs from Ansible Automation Platform do not synchronize to the automation dashboard after an upgrade, an interrupted synchronization job might be blocking the process. This occurs if the automation dashboard service stops or restarts while a synchronization task is active.

To resolve this issue, you must manually remove the stuck jobs from the database.

Prerequisites

- You have ssh access to the host machine.
- You have access to the PostgreSQL database, including the database user password defined in your inventory file (variable `dashboard_pg_password`).

Procedure

1. Connect to the automation dashboard database. You must replace `<password>` with your

configured `dashboard_pg_password`. Replace 127.0.0.1 with database server address if external database is used.

```
POSTGRES_PASSWORD=<password> psql -h 127.0.0.1 -p 5432 -U aapdashboard -d
aapdashboard
```

2. Identify jobs that are in a pending or running state:

```
SELECT * FROM scheduler_syncjob WHERE status IN ('pending','waiting','running');
```

3. Wait approximately one minute and run the command again. If the same job IDs appear in the output, these jobs are stuck.
4. Delete the stuck jobs using their ID. Replace `<id>` with the ID returned in the previous step (for example, `20, 21`):

```
DELETE from scheduler_syncjob WHERE id IN (<id>);
```

Verification

- Refresh the automation dashboard to confirm that synchronization has resumed.

1.5. Uninstalling automation dashboard

Uninstall automation dashboard and its dependencies by using a single command, ensuring a clean removal from your host.

Procedure

- Run the following command to uninstall automation dashboard and its dependencies, including the PostgreSQL database container:

```
ansible-playbook -i inventory
ansible.containerized_installer.dashboard_uninstall
```

1.6. Filter and save automation data for reporting

Automation dashboard provides filtering options to analyze your Ansible Automation Platform automation runs. You can select one or more filtering options to customize your report, select a time period and a currency, and save your report to your automation dashboard.

1.6.1. Filters

Use the following filtering options to customize your report:

- **Template:** select one or more Job Templates

- **Organization:** select one or more Organizations
- **Project:** select one or more Projects
- **Label:** select one or more automation projects by label.

NOTE	You must preconfigure and assign labels to Ansible Automation Platform for display in automation dashboard. For more information on configuring labels, see Creating a job template.

1.6.2. Time period and currency

After selecting your filters, select a time period for analysis, and select a currency to show automation savings.

- A shorter time period is useful when considering specific automation use cases.
- A longer time period is useful when considering overall platform usage and automation growth.
- Changing from one currency to another does not convert the value of that currency. You must manually change the manual and automation cost figures to reflect whichever currency you select.

1.6.3. Saving a report

Use **Save as Report** to save this report to your automation dashboard. You can retrieve it at any time by using **Select a Report**.

1.7. Summary of top and overview usage

Automation dashboard displays a summary of the top and overview usage for your selected report. This includes the following data:

- **Successful jobs**: Number of job runs that completed without error in the selected period. Use the ratio between successful and failed jobs to track automation health and reliability over time.
- **Failed jobs**: Number of job runs that ended in failure in the selected period. Review failed jobs to fix playbooks, credentials, or inventory issues and improve success rates.
- **Hosts automated**: Number of hosts that executed at least one automation job in the selected period. Indicates how much of your inventory is actively automated and can help with license or capacity planning.
- **Hours of automation**: Sum of all job runtimes in the selected period. Reflects total automation workload and can inform capacity planning and resource allocation.
- **Number of times jobs were run**: Total number of job executions in the selected period, regardless of success or failure. Use this to understand automation volume, trends, and adoption over time.
- **Number of hosts jobs are running on**: Number of hosts that ran at least one job in the selected period. Complements run count by showing how broadly automation is applied across your inventory.

- **Top 5 projects**: Projects ranked by total job count in the selected period. Helps identify which projects are driving the most automation activity.
- **Top 5 users**: Users ranked by automation runs they triggered or that ran in their context in the selected period. Shows individual adoption and activity.

NOTE	Scheduled jobs can affect these results, because they do not represent a real, logged-in user.

1.8. Analyzing costs and savings

To calculate total savings, the cost and savings analysis compares manual labor costs against Ansible Automation Platform execution costs. The dashboard prorates the monthly subscription cost into daily reporting values.

Procedure

1. In the **Monthly AAP cost** field, enter the monthly cost of running the Ansible Automation Platform. This value includes license, labor, and infrastructure costs to run Ansible Automation Platform. It is used to calculate the automation savings.
2. In the **Hourly rate for manually running the job ($)** field, enter the hourly labor cost used to estimate what it would cost to run these jobs manually. This is used to calculate manual cost and savings.
3. Optional: Select **Time taken to create automation** to include the estimated time spent creating or authoring the automation (for example, writing playbooks, setting up jobs) before it could be run.

   Automation dashboard calculates the following data points based on your inputs:

   - **Cost of manual automation**: Total cost if all jobs were run manually.
   - **Cost of automated execution**: Total cost of running jobs on Ansible Automation Platform.
   - **Total savings/cost avoided**: Difference between manual and automated cost.
   - **Total hours saved/avoided**: Time saved by automation vs manual execution.
   - **Time taken to manually execute (min)**: Estimated manual execution time in minutes.
   - **Time taken to create automation (min)**: Estimated time spent creating or authoring the automation (e.g. writing playbooks, setting up jobs) before it could be run. Included in cost when the switch above is on.

Verification

- Navigate to the Reports page to view the Total savings based on your updated cost configurations.

NOTE	You can export the cost and savings analysis as a CSV or PDF file. You must specify a date range to generate a PDF report.

1.9. Cost and savings analysis metrics

The costs and savings analysis generates the following metrics to quantify the return on investment (ROI) derived from automation execution.

Table 1. Costs and Savings Output Metrics

Metric	Description
Cost of manual automation	Total cost if all jobs were run manually. Calculated as: (Manual time in minutes × Host executions) × Average labor cost per minute.
Cost of automated execution	Total cost of running jobs on Ansible Automation Platform. Calculated as: (Running time in minutes × Prorated subscription cost per minute).
Total savings/cost avoided	The difference between manual and automated costs.
Total hours saved/avoided	Time saved by automation compared to manual execution.
Time taken to manually execute (min)	Estimated manual execution time in minutes.
Time taken to create automation (min)	Estimated time spent creating or authoring the automation (for example, writing playbooks or setting up jobs). Included in cost calculations when enabled in settings.
Hourly rate for manually running the job ($)	The hourly labor cost used to estimate the expense of running jobs manually. Used to calculate manual cost and savings.
Monthly cost of running Ansible Automation Platform	Monthly cost of running the Ansible Automation Platform. This value includes license, labor and infrastructure costs to run Ansible Automation Platform. It is used to calculate the automation savings

2. Appendix

The inventory variables required by the automation dashboard installer are described in the following table:

2.1. Inventory variables

The following variables control how automation dashboard interacts with remote hosts.

Inventory variable	Description
`aap_auth_provider_name`	Natural language name shown on the login page. Default: `Ansible Automation Platform`
`aap_auth_provider_protocol`	Enter http or https

Inventory variable	Description
`aap_auth_provider_aap_version`	Enter one value - the version of automation dashboard , valid values are 2.4, 2.5 or 2.6
`aap_auth_provider_host`	Ansible Automation Platform IP or DNS name, with optional port
`aap_auth_provider_check_ssl`	Enforce TLS check or not
`aap_auth_provider_client_id`	Ansible Automation Platform OAuth2 application `client_id`. Required for SSO authentication.
`aap_auth_provider_client_secret`	Ansible Automation Platform OAuth2 application `client_secret`
`initial_sync_days`	Requests x number of days of old data, counting from "today"
`initial_sync_since`	Requests data from the specified data until "today"
`dashboard_update_secret`	Forces regeneration of autogenerated Podman secrets. Store the password for database access by using a Podman secret. Set `dashboard_update_secret` to true if you changed the `dashboard_pg_password` in inventory.
`nginx_disable_https`	Enables use of http instead of https for automation dashboard
`nginx_http_port`	Configures the HTTP port for automation dashboard
`nginx_https_port`	Configures the HTTPS port for automation dashboard
`dashboard_tls_cert`	The public TLS certificate for the automation dashboard. This file must include the server certificate, intermediate CA certificates, and the root CA certificate. The server certificate must appear at the beginning of the file.
`dashboard_tls_key`	The private TLS key for the automation dashboard. If not specified, the installation program generates a self-signed key.
`postgresql_admin_username`	Admin username to access PostgreSQL database
`postgresql_admin_password`	Admin password to access PostgreSQL database
`registry_username`	Username used to pull container images from `registry.redhat.io`. This variable is required when using an online installation program.
`registry_password`	Password used to pull container images from `registry.redhat.io`. This variable is required when using an online installation program.
`dashboard_pg_containerized`	Configures the installation program to install and configure the database as a container on the same host as automation dashboard. `True` is also the only supported value.
`dashboard_admin_password`	The password for the automation dashboard administrator user. The username is always `admin`.

Inventory variable	Description
`dashboard_pg_host`	The hostname or IP address of the PostgreSQL database. This host must be accessible from the installation program node on port 5432. If a public IP address blocks this port, you must use the internal IP address.
`dashboard_pg_username`	The database user for automation dashboard.
`dashboard_pg_password`	The password for the `dashboard_pg_username`.
`dashboard_pg_database`	The database schema name for automation dashboard.
`bundle_install`	Indicates the required container images are already included in the installation bundle (tar file).
`bundle_dir`	The directory where the installation bundle unpacks `+ /bundle` (for example: `/home/<username>/ansible-automation-dashboard-containerized-setup/bundle`). The default value is relative to the current directory (PWD) and should work without modification.
`redis_mode`	Configures the Redis deployment mode. The required value is `standalone`.
`registry_url_aap_automation_dashboard`	Configures a custom container registry URL specifically for the automation dashboard images. Use this if you must pull dashboard images from a different registry than your database or Redis images.
`registry_ns_aap_automation_dashboard`	Configures a custom namespace specifically for the automation dashboard images.